Android Sale Item Service Guide

The Merchant Sale Item Service enables merchants to manage their sale items by offering operations to create, retrieve, update and delete items.

Below are the key utility features available:

- **Create Merchant Sale Items**: Processes the Create Merchant Sale Item request and creates the specified sale items for the merchant.
- **Get All/Specific Sale Items**: Processes the Get Merchant Sale Item request and retrieves the specified sale items for the merchant. This can be used to fetch details of a specific sale item or the complete list of sale items associated with the requested user.
- **Update Existing Sale Items**: Processes the Update Merchant Sale Item request and updates the specified sale items for the merchant.
- **Delete Merchant Sale Items**: Processes the Delete Merchant Sale Item request and then removes the requested sale item for the merchant.

Prerequisites

Before you continue, ensure you have the following prerequisites completed and ready:

- **Merchant Client Application Credentials and Access Token:** Ensure that merchant onboarding is complete and that you've received your client application credentials and access token before proceeding.
- **Access to Deluxe Developer Portal:** This portal provides additional resources, API documentation, and integration tools.
- **Merchant Authorization & Accounts Guide:** This step is required to obtain the merchant authorization token and

ON THIS PAGE

Prerequisites

Create Merchant Sale Items

Get All/Specific Sale Items

Update Existing Sale Items

Delete Merchant Sale Items

Note: *If you don't have a Deluxe Merchant Account or Access Token, simply visit Merchant Services Get Started and contact us today.*

Create Merchant Sale Items

Creates one or more sale items under the merchant's account.

Step 1: Create a MerchantSaleItemService Object

```
var merchantSaleItemService =
MerchantSaleItemService()
```

Step 2: Create a Merchant Sale Items

The CreateMerchantSaleItemRequest is used to create new sale items for a merchant account. It includes accessToken(via MerchantAccountService) and a saleItem list of MerchantSaleItem objects with details like item ID, product name, price, category, category ID, and other relevant info.

```
var saleItem = MerchantSaleItem(
saleItemId: 1234,
productName: "Veggie Burger",
price: 5.99,
category: "Main Course",
categoryId: 12,
defaultQuantity: 1,
isvariableAmount: false,
sku: "VB123",
taxID: "TAX456",
saleItemImage: "base64EncodedImageString",
isActive: true,
option1: [option1],
saleItemImageURL:
"https://example.com/images/veggie_burger.png"
)
```

Pass the request and bearerToken to the createMerchantSaleItem() function.

```
 var response =
merchantSaleItemService.createMerchantSaleItem(
        createMerchantSaleItemRequest =
merchantSaleItemRequest,
        bearerToken = token
    )
```

Step 3: Response Handling

The createMerchantSaleItem() function returns a CreateMerchantSaleItemResponse object contains the following:

- **If success = true**, the response will contain a list of created MerchantSaleItem objects, each containing details such as sale item ID, product name, price, category, category ID, and other relevant information.
- **If success = false**, indicates failure and corresponding failures reasons can be found in errorCode & message.

Get All/Specific Sale Items

Processes the Get Merchant Sale Item request and retrieves the specified sale items for the merchant. This can be used to fetch details of a specific sale item or the complete list of sale items associated with the requested user.

Step 1: Create a MerchantSaleItemService Object

```
var merchantSaleItemService =
MerchantSaleItemService()
```

Step 2: Get Merchant Sale Items

The GetMerchantSaleItemRequest includes accessToken (via MerchantAccountService), saleItemId (unique ID of the sale item), and pagination parameters page and pageSize.

```
var saleItemRequest = GetMerchantSaleItemRequest(
accessToken: "761f497a-65c4-489e-973d-
79ab84fd3011",
saleItemId: saleItemid,
page: 1,
pageSize: 10
);
```

Note: *If there is a valid saleItemId given in the request, then only that specific sale item details will be fetched, otherwise it will fetch the entire sale items for the requested merchant.*

Pass the request and bearer token to the getMerchantSaleItem() function.

```
        bearerToken = token
    )
```

The **bearerToken** should be the token obtained from the MerchantAuthorizationService.

Step 3: Response Handling

The getMerchantSaleItem() function returns a GetMerchantSaleItemResponse object which contains the following:

- **If success = true**, response will contain list of MerchantSaleItem objects each contains details such as sale item ID, product name, price, category, category ID, and other relevant information.
- **If success = false**, indicates failure and corresponding failures reasons can be found in errorCode & message.

Update Existing Sale Items

Processes the Update Merchant Sale Item request and updates the specified sale items for the merchant.

Step 1: Create a MerchantSaleItemService Object

```
var merchantSaleItemService =
MerchantSaleItemService()
```

Step 2: Create a UpdateMerchantSaleItemRequest Object

The UpdateSaleItemRequest includes accessToken (via MerchantAccountService) and a saleItem list of MerchantSaleItem objects with updated details like item ID, product name, price, category, category ID, and other relevant info.

```
var updateSaleItemRequest =
UpdateMerchantSaleItemRequest(
accessToken: "761f497a-65c4-489e-973d-
79ab84fd3011",
saleItem: 501
);
```

```
 var response =
merchantSaleItemService.updateMerchantSaleItem(
        updateMerchantTaxRequest =
updateSaleItemRequest,
        bearerToken = token
    )
```

The **bearerToken** should be the token obtained from the MerchantAuthorizationService.

Step 3: Response Handling

The updateMerchantSaleItem() function returns a UpdateMerchantSaleItemResponse object which contains the following:

- **If success = true**, response will contain list of updated MerchantSaleItem objects each contains details such as sale item ID, product name, price, category, category ID, and other relevant information.
- **If success = false**, indicates failure and corresponding failures reasons can be found in errorCode & message.

Delete Merchant Sale Items

Processes the Delete Merchant Sale Item request and then removes the requested sale item for the merchant.

Step 1: Create a MerchantSaleItemService Object

```
var merchantSaleItemService =
MerchantSaleItemService()
```

Step 2: Delete a Merchant Sale Item

The DeleteMerchantSaleItemRequest includes accessToken (via MerchantAccountService) and saleItemId, the unique identifier of the sale item to be deleted.

```
var deleteSaleItemRequest =
DeleteMerchantSaleItemRequest(
accessToken: "761f497a-65c4-489e-973d-
79ab84fd3011",
saleItemId: saleItemId
);
```

```
 var response =
merchantSaleItemService.deleteMerchantSaleItem(
        deleteMerchantSaleItemRequest =
deleteSaleItemRequest,
        bearerToken = bearerToken
    )
```

The **bearerToken** should be the token obtained from the MerchantAuthorizationService.

Step 3: Response Handling

The deleteMerchantSaleItem() function returns a DeleteMerchantSaleItemResponse object which contains the following:

- **If success = true**, response will contain list of MerchantSaleItem object which is deleted.
- **If success = false**, indicates failure and corresponding failures reasons can be found in errorCode & message.

Note: *Please refer to the Complete SDK Reference for additional details Android SDK Reference Guide*

deluxe. connect

Payments API